

Black Coordinates List

1. רקע מבצעי

בשנים האחרונות מתבצעת פעילות סייבר עוינת מתמשכת נגד גופים ממשלתיים, תשתיות רגישות (כגון הרכבת הקלה בגוש דן) ואישים בעלי חשיבות. פעילות זו מתבצעת באמצעות תשתיות מבוזרות, שימוש בכתובות IP מתחלפות והסתרת מקורות הפעולה מאחורי שכבות טכנולוגיות שונות.

היקף הפעילות, התדירות והנזק הפוטנציאלי הובילו להבנה כי התמודדות עם האיום אינה יכולה להישאר ברמה הדיגיטלית בלבד. נדרש חיבור עמוק יותר בין מידע טכנולוגי לבין המציאות הפיזית שמאחוריו.

במוסד התקבלה החלטה לפתח יכולת תשתיתית שתאפשר ראייה רחבה יותר של הזירה — כזו שתשמש בסיס לפעילות מודיעינית, חקירתית ומבצעית בהמשך.

בשלב זה, לאחר שהוגדר הצורך המבצעי והובהרה חשיבותו, אתם נקראתם למשימה.

עליכם להקים את **Black Coordinates List** — מערכת תשתיתית שתיבנה בצורה מדויקת, יציבה וניתנת להרחבה, ותשמש רכיב ליבה במערך גדול יותר של מערכות ויכולות.

הדגש בפרויקט זה אינו על תחכום או אלגוריתמיקה, אלא על בנייה נכונה של תשתית: הבנה של זרימת מידע, הפרדת אחריות, ויכולת לסמוך על המערכת כבסיס להמשך פעילות.

2. מה המערכת עושה

מערכת **Black Coordinates List** היא מערכת תשתיתית שמטרתה לבצע את הפעולות הבאות:

- קבלת כתובות IP הקשורות לפעילות סייבר
- המרת כתובות IP לנקודות ציון גאוגרפיות באמצעות שירות חיצוני
- העברת נקודות הציון בין שירותים פנימיים במערכת
- אחסון נקודות הציון במאגר נתונים מהיר מבוסס Key-Value

- שליפה מרוכזת של כלל נקודות הציון שנשמרו במערכת

המערכת פועלת כ־**שכבת ליבה תשתיתית**, ומספקת נתונים לשימוש עתידי על ידי מערכות, שירותים או גורמים אחרים בארגון.

3. ארכיטקטורה כללית של המערכת

מערכת **Black Coordinates List** בנויה כמערכת תשתיתית מבוססת שירותים (Services), שבה כל רכיב אחראי על תחום מוגדר וברור.

המערכת עושה שימוש בתקשורת HTTP בין שירותים, בשירות חיצוני להמרת IP למיקום גאוגרפי, ובמסד נתונים מהיר מסוג Key-Value לצורך אחסון ושליפה.

הטכנולוגיות המרכזיות במערכת:

- Python + FastAPI עבור שירותי ה־Backend
- HTTP (REST) לתקשורת בין רכיבים
- שירות חיצוני להמרת IP לנקודות ציון
- Redis כמאגר נתונים תשתיתי מהיר
- עבודה בסביבה מבודדת (Container-based)

הארכיטקטורה מתוכננת כך שכל רכיב ניתן להבנה, לבדיקה ולהחלפה, מבלי להשפיע על שאר המערכת.

רכיבי המערכת

המערכת מורכבת משלושה רכיבים מרכזיים:

1. שירות קליטת IP והמרה למיקום (Service A)
2. שירות אחסון ושליפה של נקודות ציון (Service B)
3. מאגר נתונים Redis

להלן פירוט אחריות מדויק לכל רכיב.

3.1 שירות IP Resolution Service – A

תפקיד במערכת

שירות זה מהווה את נקודת הכניסה למידע חיצוני. הוא אחראי לקבל כתובות IP, להמיר אותן למיקומים גאוגרפיים, ולהעביר את הנתונים הלאה לשירות פנימי.

מה השירות עושה

- חושף API לקבלת כתובת IP
- שולח בקשה לשירות חיצוני להמרת IP למיקום
- מקבל ומעבד את התשובה של השירות החיצוני (נקודות ציון)
- שולח את נקודות הציון לשירות האחסון (**שירות B** עליו מפורט בהמשך)

מה השירות לא עושה

- אינו שומר נתונים
- אינו עובד ישירות מול Redis
- אינו מבצע שליפות של מידע
- אינו מקבל החלטות או מנתח נתונים
- אינו אחראי על הצגת מידע

זהו שירות מתאם (Adapter) בין העולם החיצוני לבין המערכת הפנימית.

3.2 שירות Coordinates Storage Service – B

תפקיד במערכת

שירות זה מהווה את שכבת האחסון והשליפה של המערכת. הוא אחראי לקבל נקודות ציון, לשמור אותן בצורה מסודרת, ולהחזיר אותן לפי דרישה.

מה השירות עושה

- חושף API פנימי לקבלת נקודות ציון (**משירות A**)
- מבצע ולידציה בסיסית לנתונים הנכנסים באמצעות pydantic

- שומר את הנתונים ב-Redis
- מאפשר שליפה מרוכזת של כלל הנתונים שנשמרו

מה השירות לא עושה

- אינו פונה לשירותים חיצוניים
- אינו יודע מה מקור הנתונים (מאיזה שרת הגיעו, משתמש, מערכת)
- אינו מבצע חישובים או ניתוחים

זהו שירות תשתיתי "שקט", שמטרתו אחת: אחסון ושליפה אמינים.

Redis 3.3 – מאגר הנתונים

תפקיד במערכת

Redis משמש כמאגר נתונים מהיר לאחסון נקודות הציון.

מאפיינים מרכזיים

- מבוסס Key-Value
- In-Memory עם Persistence בסיסי
- גישה מהירה ופשטה
- מתאים לאחסון מבנים פשוטים של נתונים

אופן השימוש בפרויקט

- שמירת נקודות ציון כמבני נתונים פשוטים
- שליפה כללית של כל הערכים
- ללא שימוש ב-TTL דינמי
- ללא לוגיקה עסקית בתוך מסד הנתונים

עיקרון ארכיטקטוני מנחה

כל רכיב במערכת:

- יודע רק את המינימום הנדרש
- אינו תלוי בלוגיקה פנימית של רכיבים אחרים

מבנה זה מאפשר:

- פשטות
- יכולת בדיקה
- תחזוקה קלה
- הרחבה עתידית ללא שבירת המערכת

4. זרימת מידע מלאה של המערכת :

זרימה ראשית: קליטת IP ואחסון נקודות ציון

1. גורם חיצוני (סוכן, מערכת או כלי בדיקה) שולח בקשת **POST** ל-Service A עם כתובת IP.
2. Service A מקבל את הבקשה ומתחיל תהליך טיפול בכתובת ה-IP.
3. Service A שולח בקשת HTTP לשירות חיצוני להמרת IP למיקום גאוגרפי.
4. השירות החיצוני מחזיר ל-Service A נתונים הכוללים נקודות ציון גאוגרפיות.
5. Service A מבצע עיבוד בסיסי לנתונים שהתקבלו מהשירות החיצוני, ומחלץ את המידע הרלוונטי בלבד.
6. לאחר קבלת נתונים תקינים, Service A שולח בקשת **POST** פנימית ל-Service B עם נקודות הציון.
7. Service B מקבל את הבקשה ומבצע ולידציה בסיסית לנתונים.
8. Service B שומר את נקודות הציון במאגר הנתונים Redis.
9. התהליך מסתיים לאחר שהנתונים נשמרו בהצלחה במאגר.

זרימה משנית: שליפת נתונים

1. גורם פנימי (כלי בדיקה, מערכת עתידית או שירות נוסף) שולח בקשת **GET** ל-Service B.
2. Service B מקבל את הבקשה ומבצע שליפה של כלל נקודות הציון ממאגר הנתונים.
3. Redis מחזיר את הנתונים שנשמרו.

4. Service B מחזיר את רשימת נקודות הציון, בהתאמה לכתובת ה-IP הרלוונטית, לגורם המבקש.

דגשים חשובים לזרימת המידע

- כל זרימה מתחילה בבקשת HTTP מפורשת.
- Service A אינו שומר נתונים ואינו ניגש ל-Redis.
- Service B אינו פונה לשירותים חיצוניים ואינו יודע מה מקור הנתונים.

6. חלוקת עבודה ואחריות בין מפתחים

המערכת **Black Coordinates List** מפותחת על ידי צוות קטן, כאשר האחריות מחולקת בצורה ברורה בין שני מפתחים.

החלוקה נועדה לאפשר עבודה מקבילית, אחריות ברורה, וצמצום תלות בין רכיבים.

כל מפתח אחראי על שירות אחד, כולל הפיתוח, הבדיקות והאינטגרציה שלו עם שאר המערכת.

מפתח א' – Service A (IP Resolution Service)

תחום אחריות

מפתח א' אחראי על כל הקשור לקליטת כתובות IP ותקשורת עם העולם החיצוני.

אחריות עיקרית

- פיתוח Service A באמצעות FastAPI
- הגדרת Endpoint לקליטת כתובת IP
- מימוש קריאה לשירות חיצוני להמרת IP למיקום גאוגרפי
- עיבוד בסיסי של הנתונים המתקבלים
- שליחת נקודות הציון ל-Service B באמצעות בקשת HTTP פנימית
- טיפול בשגיאות תקשורת ושגיאות נתונים
- בדיקות ידניות של הזרימה מקצה לקצה עד שליחת הנתונים

גבולות אחריות

- אינו שומר נתונים
- אינו עובד ישירות מול Redis
- אינו מבצע שליפות
- אינו אחראי על מבנה האחסון

מפתח ב' – Service B (Coordinates Storage Service)

תחום אחריות

מפתח ב' אחראי על שכבת האחסון והשליפה של המערכת.

אחריות עיקרית

- פיתוח Service B באמצעות FastAPI
- הגדרת Endpoint לקבלת נקודות ציון מ-Service A
- ולידציה בסיסית של הנתונים הנכנסים
- חיבור ל-Redis באמצעות משתני סביבה
- שמירת נקודות הציון במבנה נתונים מוסכם
- מימוש Endpoint לשליפה מרוכזת של כלל הנתונים
- בדיקות של אחסון, שליפה והתנהגות Redis

גבולות אחריות

- אינו פונה לשירותים חיצוניים
- אינו יודע מה מקור הנתונים (IP, משתמש, מערכת)
- אינו מבצע חישובים או ניתוחים
- אינו אחראי על זרימת הנתונים לפני האחסון

אחריות משותפת

שני המפתחים אחראים במשותף על:

- תיאום מבנה הנתונים שעובר בין השירותים
- הגדרת חוזה ברור בין Service A ל-Service B
- בדיקות אינטגרציה בין השירותים
- שמירה על קוד ברור, קריא וניתן לתחזוקה

חלוקת עבודה זו מדמה עבודה בצוות Backend אמיתי, שבו כל מפתח אחראי על רכיב מוגדר, אך מבין את הזרימה הכוללת של המערכת.

7. סטנדרט עבודה עם Git ואסטרטגיית Branches

מבנה Repository כללי

- Repository אחד לכל המערכת
- כל שירות בתיקייה נפרדת
- קובץ README המסביר בקצרה את מבנה הפרויקט

Branching Strategy

העבודה מתבצעת באמצעות אסטרטגיית Branches ברורה:

- **main**
 - מכיל קוד יציב בלבד (שעובד בפועל לאחר בדיקות)
 - אין עבודה ישירה על ענף זה
- **dev**
 - ענף הפיתוח הראשי
 - כל השינויים מתמזגים אליו לאחר אישור
- ***/feature**

- ענפי עבודה למשימות ספציפיות
- כל משימה מפותחת בענף נפרד

דוגמאות לשמות Branches

- `feature/service-a-ip-resolution`
- `feature/service-b-storage`
- `feature/redis-connection`

אופן העבודה

1. יוצרים Branch חדש מ-`dev`
2. עובדים רק עליו
3. מבצעים Commit-ים קטנים וברורים
4. פותחים Pull Request ל-`dev`
5. לאחר אישור — מבצעים Merge

אין לבצע Merge ישיר ללא Pull Request (הכוללת בקרה של חבר הצוות השני, כמפורט להלן).

Pull Requests ו-Code Review (חובה)

כל Pull Request חייב לעבור Code Review על ידי מפתח אחר בצוות.

- אין אישור עצמי
- כל מפתח מבצע Review לקוד של המפתח השני
- רק לאחר Review ואישור מפורש ניתן לבצע Merge

קוד שלא עבר Code Review — לא נכנס למערכת.

קונבנציה לכתיבת Commit Messages

כל Commit מתאר שינוי אחד ברור.

מבנה מומלץ:

`<type>: <short description>`

סוגי Commit נפוצים:

- `feat`: הוספת פונקציונליות
- `fix`: תיקון באג
- `refactor`: שינוי מבני ללא שינוי התנהגות
- `docs`: תיעוד
- `chore`: תחזוקה או קונפיגורציה

דוגמאות:

- `feat: add IP resolution endpoint`
- `fix: handle invalid IP input`
- `refactor: separate service logic from route`
- `docs: update README`

דגשים חשובים

- Commit אחד = שינוי אחד
- לא דוחפים קוד שבור
- לא מערבבים כמה נושאים באותו Commit
- כל מפתח אחראי על איכות הקוד שהוא מכניס

עבודה לפי סטנדרט זה היא חלק בלתי נפרד מהמשימה ומדמה תהליך עבודה אמיתי בצוותי פיתוח מקצועיים.

8. מבנה תיקיות וקבצים

מבנה הפרויקט מחולק לפי שירותים, כאשר כל שירות מבודד וברור, ומכיל רק את הקבצים הנדרשים לאחריות שלו.

המבנה נועד:

- לאפשר עבודה מקבילית בין מפתחים
- לשמור על הפרדת אחריות
- להיות ברור גם למי שקורא את הפרויקט בפעם הראשונה

מבנה כללי של הפרויקט

```
black-coordinates-list/
|
├── service-a/
|   ├── app/
|   │   ├── main.py
|   │   ├── routes.py
|   │   ├── services.py
|   │   ├── Dockerfile
|   │   └── schemas.py
|   └── requirements.txt
|
├── service-b/
|   ├── app/
|   │   ├── main.py
|   │   ├── routes.py
|   │   ├── storage.py
|   │   ├── Dockerfile
|   │   └── schemas.py
|   └── requirements.txt
|
├── docker-compose.yml
├── .env.example
└── README.md
```

הסבר מבנה – ברמת צוות מקצועי

תיקיית שורש (Root)

- `/black-coordinates-list`
מכילה את כל רכיבי המערכת.
- `docker-compose.yml`
מאפשר הרצה מקומית של כלל השירותים וה-Redis.

- `env.example.`
דוגמה למשתני סביבה (לא מכיל סודות).
- `README.md`
תיאור כללי של הפרויקט, הארכיטקטורה ואופן ההרצה.

Service A – IP Resolution Service

```
service-a/  
├── app/  
│   ├── main.py  
│   ├── routes.py  
│   ├── services.py  
│   ├── Dockerfile  
│   └── schemas.py
```

חלוקת אחריות:

- `main.py`
יצירת אפליקציית FastAPI וחיבור ה-`routes`.
- `routes.py`
נקודות הקצה (Endpoints) בלבד.
ללא לוגיקה מורכבת.
- `services.py`
לוגיקת העבודה:
 - קריאה לשירות החיצוני
 - עיבוד נתונים
 - שליחה ל-Service B
- `schemas.py`
מודלים של בקשות ותשובות (Pydantic).

Service B – Coordinates Storage Service

```
service-b/
```

```
├── app/
│   ├── main.py
│   ├── routes.py
│   ├── storage.py
│   ├── Dockerfile
│   └── schemas.py
```

חלוקת אחריות:

- `main.py`
יצירת אפליקציית FastAPI וחיבור ה-`routes.py`.
- `routes.py`
נקודות קצה:
 - קבלת IP עם נקודות ציון
 - שליפת נתונים
- `storage.py`
עבודה מול Redis:
 - חיבור
 - שמירה
 - שליפה
- `schemas.py`
מודלי נתונים (Pydantic).

קווים מנחים חשובים

- אין קוד לוגיקה בתוך `routes.py`
- כל קובץ אחראי על תפקיד מוגדר וברור
- אין תלות ישירה בין Service A ל-Redis
- התקשורת בין שירותים מתבצעת רק דרך HTTP

9. דרישות פריסה – Kubernetes (Minikube)

המערכת **Black Coordinates List** חייבת להיפרס על גבי Kubernetes בסביבת **Minikube** מקומית.

דרישות כלליות לפריסה:

- כל רכיבי המערכת ירוצו כ־Pods בתוך Kubernetes
- כל שירות Backend יוגדר כך שניתן יהיה להריץ אותו מחדש ללא תלות במופע ספציפי
- רכיב האחסון יוגדר כך שהמידע שבו אינו תלוי בהרצה חד־פעמית
- התקשורת בין רכיבי המערכת תתבצע באמצעות Kubernetes Services בלבד
- לפחות שירות אחד יהיה נגיש מהמחשב הלוקאלי לצורכי בדיקות

הפריסה תתבצע באמצעות קבצי YAML ותנוהל דרך הטרמינל.

הרחבה אופציונלית

סטודנטים המעוניינים בכך יוכלו, בהמשך, לפרוס את אותה מערכת גם בסביבת Kubernetes מנוהלת (כגון OpenShift),
תוך שימוש באותם עקרונות ובקבצי YAML דומים.